

WEEK 2 – SQL FOR DATA ANALYSIS

Sales Data Analysis

Data Analyst Internship – Week 2 Submission

Project Detail	Value
Dataset	SQL Sales Dataset – 200 Rows
Records Analyzed	200
Date Range	24 Dec 2024 – 24 Jun 2025
SQL Topics	SELECT, WHERE, GROUP BY, ORDER BY, SUM, AVG, COUNT, JOIN, Subquery, CAS
Tools	Excel + SQL (SQLite-compatible queries)

E

This submission demonstrates practical SQL analysis on the supplied sales dataset, including customer, category, regional, and order-value analysis.

1. Dataset Overview

The supplied dataset contains **200 sales records** and 10 fields. The order dates range from **24 Dec 2024** to **24 Jun 2025**.

Column	Description
order_id	Unique order identifier
customer_name	Customer associated with the order
order_date	Date of the order
category	Product category
sub_category	Product sub-category
product_name	Product name
quantity	Units ordered
unit_price	Price per unit
total_price	Total order value
region	Sales region

Week 2 Requirements

The Week 2 brief covers basic SQL queries (SELECT, WHERE, GROUP BY, ORDER BY), aggregations (SUM, AVG, COUNT), joins, subqueries, and CASE statements, with analysis such as top customers and average order values.

2. Overall Sales Analysis

Metric	Result
Total Orders	200
Total Sales	2,420,107
Average Order Value	12,100.53
Highest Order Value	47,940
Lowest Order Value	79

SQL Query

```
SELECT COUNT(*) AS total_orders, SUM(total_price) AS total_sales, ROUND(AVG(total_price), 2) AS  
average_order_value, MAX(total_price) AS highest_order_value, MIN(total_price) AS lowest_order_value FROM sales;
```

The dataset contains 200 orders with total sales of 2,420,107. The average order value is 12,100.53, while the highest and lowest order values are 47,940 and 79, respectively.

3. Basic SQL Queries

SELECT – Retrieve all records

```
SELECT * FROM sales;
```

WHERE – Identify orders above 20,000

```
SELECT * FROM sales WHERE total_price > 20000;
```

ORDER BY – Sort orders from highest to lowest value

```
SELECT order_id, customer_name, total_price FROM sales ORDER BY total_price DESC;
```

These queries demonstrate retrieving, filtering, and sorting records using basic SQL clauses.

4. Top Customers

```
SELECT customer_name, SUM(total_price) AS total_spent FROM sales GROUP BY customer_name ORDER BY total_spent DESC  
LIMIT 10;
```

Rank	Customer	Total Spent
1	Lynn Garrison	47,940
2	Debbie Turner	44,990
3	Megan Charles	44,620
4	Michelle Beltran	42,471
5	Rick Sanford	41,211
6	Patrick Watson	37,680
7	Christopher Meyer	37,320
8	Todd Barnes	36,152
9	Stephanie Martinez	34,576
10	Kimberly Alvarado	31,752

The query groups sales by customer, calculates total spending, and sorts customers in descending order. The table above shows the ten customers with the highest aggregated spending in the supplied dataset.

5. Category Analysis

```
SELECT category, COUNT(*) AS total_orders, SUM(total_price) AS total_sales, ROUND(AVG(total_price), 2) AS  
average_order_value FROM sales GROUP BY category ORDER BY total_sales DESC;
```

Category	Orders	Total Sales	Avg Order Value
Furniture	59	714,399	12,108.46
Grocery	49	672,147	13,717.29
Electronics	52	549,302	10,563.5
Clothing	40	484,259	12,106.48

Furniture has the highest total sales in the dataset at 714,399. The category analysis combines COUNT, SUM, AVG, GROUP BY, and ORDER BY in a single business-oriented query.

6. Regional Analysis

```
SELECT region, COUNT(*) AS total_orders, SUM(total_price) AS total_sales, ROUND(AVG(total_price), 2) AS  
average_order_value FROM sales GROUP BY region ORDER BY total_sales DESC;
```

Region	Orders	Total Sales	Avg Order Value
South	66	808,651	12,252.29
East	51	619,250	12,142.16
West	43	599,367	13,938.77
North	40	392,839	9,820.98

South has the highest total sales among the regions at 808,651. South also has 66 orders in the dataset.

7. JOIN Analysis

A small category reference table is used to demonstrate a JOIN between the sales table and a lookup table.

```
CREATE TABLE category_info ( category TEXT, category_type TEXT ); INSERT INTO category_info (category,  
category_type) VALUES ('Electronics', 'Technology'), ('Furniture', 'Home'), ('Grocery', 'Daily Essentials'),  
('Clothing', 'Fashion'); SELECT s.order_id, s.product_name, s.category, c.category_type, s.total_price FROM sales  
s JOIN category_info c ON s.category = c.category;
```

The JOIN matches the category field in the sales table with the category field in the reference table, allowing additional category information to be returned with each sales record.

8. Subquery Analysis

```
SELECT order_id, customer_name, total_price FROM sales WHERE total_price > ( SELECT AVG(total_price) FROM sales )  
ORDER BY total_price DESC;
```

The subquery calculates the overall average order value of 12,100.53. The outer query then returns orders whose total value is greater than that average. There are **81** above-average orders in the supplied dataset.

Highest Above-Average Orders

Order ID	Customer	Total Price
1125	Lynn Garrison	47,940
1019	Debbie Turner	44,990
1168	Megan Charles	44,620
1160	Michelle Beltran	42,471
1120	Rick Sanford	41,211
1070	Patrick Watson	37,680
1190	Christopher Meyer	37,320
1149	Todd Barnes	36,152
1114	Stephanie Martinez	34,576
1174	Kimberly Alvarado	31,752

9. CASE Statement Analysis

```
SELECT order_id, customer_name, total_price, CASE WHEN total_price >= 30000 THEN 'High Value' WHEN total_price >= 15000 THEN 'Medium Value' ELSE 'Low Value' END AS order_value_category FROM sales;
```

Order Value Classification

Category	Number of Orders
High Value	12
Medium Value	54
Low Value	134

The CASE statement classifies orders into High Value, Medium Value, and Low Value based on total order value. This demonstrates conditional logic within SQL.

10. Final Findings

- The dataset contains **200** sales orders.
- Total sales amount to **2,420,107**.
- The average order value is **12,100.53**.
- The highest-value order is **47,940**.
- The lowest-value order is **79**.
- **Furniture** has the highest total category sales at **714,399**.
- **South** has the highest regional sales at **808,651**.
- The top-customer analysis identifies customers by aggregated spending.
- The subquery identifies orders above the overall average, while CASE categorizes order values.

Conclusion

This project demonstrates the practical use of SQL for sales data analysis. The analysis applies filtering, sorting, aggregation, grouping, joins, subqueries, and conditional logic to derive customer, category, regional, and order-value insights from the supplied dataset.

End of Week 2 Submission